Programming Fundamentals

Week 8 — Wednesday: Debugging Python in VS Code

Programming Fundamentals Team

SETU

2026-11-09

Today — Debugging Like a Detective

By the end you should be able to:

- launch a Python file in the **VS Code debugger**
- set and remove breakpoints
- use **Continue**, **Step Over** and **Step Into**
- inspect variables, loops and conditions
- distinguish **logic errors** from **exceptions**
- find a file-path mistake using a traceback

Today is practical: predict → pause → inspect → fix → retest.

Warm-up — The Program Runs, But Is It Right?

A program can finish without a red error message and still give the **wrong answer**.

Question: If a program says the total of 10, 20 and 30 is 30, has Python crashed?

Warm-up — The Program Runs, But Is It Right?

A program can finish without a red error message and still give the **wrong answer**.

Question: If a program says the total of 10, 20 and 30 is 30, has Python crashed?

No. That is a **logic error**. We must inspect the calculation rather than add `try`/`except`.

Our Debugging Routine

1. **Reproduce:** run the same input again.
2. **Predict:** write down what should happen.
3. **Pause:** add a breakpoint just before the suspicious line.
4. **Inspect:** look at variable values and the next statement.
5. **Step:** follow one instruction at a time.
6. **Fix and retest:** change one thing; try another input.

Don’t change random lines until the output looks right.

VS Code — Before You Start

1. Open the `Python-Examples` folder using **File → Open Folder**.
2. Open `01_running_total.py`.
3. Make sure the **Python** extension and a Python interpreter are available.
4. Click beside a line number to add a red **breakpoint**.
5. Press **F5** → choose **Python Debugger** / **Python File** if prompted.

Watch: the yellow execution marker shows the next line to run.

Debugger Controls — Your Remote Control

Action	Shortcut	What happens
Continue	`F5`	Run to the next breakpoint
Step Over	`F10`	Run the current line; don't enter a function call
Step Into	`F11`	Enter the function being called
Step Out	`Shift+F11`	Finish the function and return to the caller
Stop	`Shift+F5`	End the debugging session

Default shortcuts; they may differ on some computers.

What Can We Inspect?

When execution is paused, explore the **Run and Debug** panel:

- **Variables → Locals**: current values, such as `total` and `price`
- **Watch**: expressions you want to monitor, e.g. `total + price`
- **Call Stack**: which functions led to the current line
- **Debug Console**: inspect expressions while paused

Important: a breakpoint pauses **before** the highlighted line executes.

Demo 1 — A Total That Forgets

Open: `01_running_total.py`

```
# Expected total: 60. Actual: 30.
def calculate_total(prices):
    total = 0
    for price in prices:
        total = price
    return total

print("Total:", calculate_total([10, 20, 30]))
```

Expected: 60; actual 30.

Debug: Breakpoint at `total = price`; step through all three iterations. What is overwritten?

Demo 1 — What Should We See?

Iteration	price	Correct running total
1	10	10
2	20	30
3	30	60

Ask: Which assignment would **add** to the total rather than replace it?

Live code: fix the line, then retest `[]`, `[10]`, `[10, 20, 30]`.

Demo 2 — Discount or Surcharge?

Open: `02_discount.py`

```
# Expected final price: 45.0. Actual: 55.0.
def apply_discount(price, percent):
    discount = price * percent / 100
    return price + discount

print("Final price:", apply_discount(50, 10))
```

Expected: 45.0; actual 55.0.

Debug: Break on `return`. Inspect `price` and `discount`. Which operator is wrong?

Demo 3 — The 18th Birthday

Open: `03_age_category.py`

```
# Expected: age 18 is Adult. Actual: Teenager.
age = 18
if age > 18:
    print("Adult")
else:
    print("Teenager")
```

Expected: Adult for age 18.

Debug: Pause on the `if`. Inspect `age`; predict `age > 18`. What happens at the boundary?

Demo 4 — Counting Passes

Open: `04_count_passes.py`

```
# Expected: 3 passes. Actual: 2.
marks = [40, 50, 75, 32]
passes = 0
for mark in marks:
    if mark > 40:
        passes += 1
print("Passes:", passes)
```

Expected: 3 passes (40, 50, 75).

Debug: Inspect `mark` and `passes`. Is the pass threshold inclusive?

Demo 5 — The Missing Last Name

Open: `05_off_by_one.py`

```
# Expected to print all four names. Actual: final name missing.
names = ["Ava", "Ben", "Cara", "Dara"]
for index in range(len(names) - 1):
    print(names[index])
```

Expected: All four names printed.

Debug: Watch `index` as the loop runs. What values does `range(len(names) - 1)` produce?

Demo 6 — A Search That Gives Up

Open: `06_search_first_match.py`

```
# Expected: True, because 42 is in the list. Actual: False.
def contains_number(numbers, target):
    for number in numbers:
        if number == target:
            return True
        else:
            return False
    return False

print("Found:", contains_number([10, 20, 42], 42))
```

Expected: True because 42 is present.

Debug: Step Into `contains_number`. Why does the function return after the first item?

Pause and Discuss — Return vs Loop

A `return` statement ends the **whole function** immediately.

If we want to search a list, when should we return `False`?

- After checking the **first** non-matching item?
- Or after checking **every** item without finding a match?

Try: move the `return False` and test `[42]`, `[10, 42]`, `[]`.

Demo 7 — The File That Disappeared

Open: `07_misspelt_filename.py`

```
# The actual file is data/students.json.
# Debug the path before deciding how to handle the exception.
import json
from pathlib import Path

folder = Path(__file__).resolve().parent
file_path = folder / "data" / "student.json"  # Spot the typo!
print("Trying:", file_path)
with open(file_path, "r", encoding="utf-8") as file:
    students = json.load(file)
print("Students loaded:", len(students))
```

Expected: Load 2 students from `data/students.json`.

Debug: Read the traceback; inspect `file_path`. Compare the filename with the Explorer panel.

File Not Found — Debug or Handle?

The program tries to open `data/student.json` but the real file is `data/students.json`.

First: fix the spelling mistake in the source code.

Then discuss: What if the file is genuinely missing on another computer?

- **Debugging** fixes the incorrect path.
- **Exception handling** can report a genuine missing file gracefully.

Do not hide a known filename typo with `except: pass`.

Demo 8 — A Dictionary Key Mismatch

Open: `08_wrong_dictionary_key.py`

```
# Expected: 12. Actual: 0.
stock = {"pens": 12, "pencils": 8}
item = "pens"
quantity = stock.get("pen", 0)
print("Quantity of", item, ":", quantity)
```

Expected: 12 pens.

Debug: Inspect `stock`, `item` and the key passed to `.get()`. Why does it silently return zero?

Demo 9 — A Boolean Surprise

Open: `09_boolean_logic.py`

```
# Expected: no discount for a 16-year-old without membership.
# Actual: discount applied.
age = 16
member = False
if age >= 18 or not member:
    print("Discount applies")
else:
    print("No discount")
```

Expected: No discount for a 16-year-old non-member.

Debug: Inspect `age`, `member` and each side of `or`. Which logical operator expresses the rule?

Exception Handled but still gives error

```
numerator = 100
denominator = 0

try:
    result = numerator / denominator
    print("Result:", result)

except ZeroDivisionError:
    print("Error: You cannot divide by zero.")

finally:
    print("Trying the calculation one more time...")
    result = numerator / denominator
    print("Final result:", result)
```

What happens?

Demo 10 — A Function Uses the Wrong Value

Open: `10_function_parameters.py`

```
# Expected area: 24. Actual: 36.
def rectangle_area(width, height):
    return width * width

print("Area:", rectangle_area(6, 4))
```

Expected: 24 (6 × 4).

Debug: Step Into the function. Compare `width`, `height` and the return expression.

An Import Does Not Copy the Code

```
main.py
   |
   | import calculator
   v
calculator.py
   |
   +-- add(a, b)
   +-- divide(a, b)
```

The function stays defined in `calculator.py`. `main.py` calls it through the imported module.

Live demo: Put a breakpoint inside `calculator.add()` and press **Step Into (F11)** from `main.py`.

Common Import Mistakes

Mistake	What to investigate
`import calculater`	Spelling: the file is `calculator.py` → `ModuleNotFoundError`
`calculator.subtract(3, 1)`	Does the module define `subtract`? → `AttributeError`
Module is in another folder	Check project structure and Python import path
File named `random.py`	It can shadow Python's standard library `random` module

Tip: Avoid giving your own files the same names as standard library modules.

Debugging Across Two Files

Open `Examples/04_debug_import/main.py`.

1. Predict the output for a 20% discount on €50.
2. Set a breakpoint on the function call in `main.py`.
3. Press **F11** to step into `pricing.py`.
4. Inspect `price`, `discount_percent` and `discount`.
5. Find the logic error and retest.

The debugger can follow execution into your own imported files.

When Do We Need `try` / `except`?

Problem	Type	Best approach
Wrong total	Logic error	Debug and correct the calculation
Wrong `if` condition	Logic error	Debug and correct the condition
Misspelt filename	Coding mistake causing `FileNotFoundError`	Correct the path
File genuinely absent	Runtime exception	Handle or stop safely

An exception handler is not a substitute for fixing faulty code.

Debugging Checklist — Before Asking for Help

- Can you reproduce the issue?
- What did you expect? What actually happened?
- What line did the debugger reach?
- What values did the variables contain?
- Did you check the **first** and **last** item in a loop?
- Did you test boundary values such as 18 or 40?
- For files: is the **path and filename** correct?

Bring your prediction and what you observed.

Week 8 — What Have We Learned?

- A program can run successfully and still be wrong.
- Breakpoints pause execution **before** a line runs.
- **Step Over** follows the next statement; **Step Into** enters a function.
- The Variables panel shows what the program actually knows.
- Boundary conditions, early returns and wrong operators are common bugs.
- Fix coding mistakes; handle genuine expected failures appropriately.

Next time: these skills will help when we introduce classes and objects.